

EDB PostgreSQL 15 (~9 TB) to Azure Database for PostgreSQL Flexible Server

Minimal-Downtime Migration Options, Architecture, Risks, and Practical Runbook

This guide summarizes reliable alternatives to dump-and-restore for migrating a large EDB PostgreSQL 15 database to Azure Database for PostgreSQL Flexible Server. It focuses on online migration, logical replication, large-scale operational risks, validation, and cutover planning.

1. Executive Recommendation

For a ~9 TB PostgreSQL database with a tight downtime requirement, a traditional `pg_dump/pg_restore` approach should not be the primary production migration method. The preferred approach is Azure Database for PostgreSQL Flexible Server Migration Service in online mode, using an initial bulk copy followed by logical decoding/CDC to keep the Azure target synchronized until cutover.

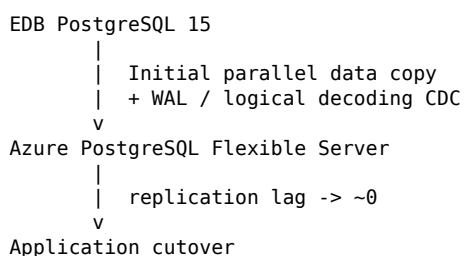
Native PostgreSQL logical replication is a strong fallback when more control is required. `pglogical` is another option when native logical replication is too restrictive. Azure DMS Classic should not be the preferred path for this 9 TB migration because Microsoft's PostgreSQL online DMS documentation describes a 1 TB per-service limitation and Microsoft now directs customers toward the migration service integrated with Flexible Server.

2. Migration Option Comparison

Option	Downtime	9 TB suitability	Complexity	Recommendation
Azure PostgreSQL Migration Service - Online	Minutes / cutover	Good; benchmark throughput	Medium	Preferred
Native logical replication	Minutes	Good	High	Strong fallback
<code>pglogical</code>	Minutes	Good	High	More flexibility
Azure DMS Classic	Minutes	Poor for 9 TB	Medium	Avoid
<code>pg_dump</code> / <code>pg_restore</code>	Hours to days	Possible	Low	If downtime acceptable
Physical replication / base backup	Very low	Excellent technically	N/A	Not direct to Flexible Server

3. Preferred Option: Azure PostgreSQL Flexible Server Migration Service - Online

The Azure-native online migration model performs a parallel initial data copy and then follows source changes through PostgreSQL logical decoding/CDC. Applications can continue writing during most of the migration. At cutover, application writes are stopped, replication is allowed to catch up, final validation is performed, and application connectivity is switched to Azure.



Microsoft documentation describes `pgcopydb`-related capabilities for data movement and logical decoding for online migrations. Validate the exact supported source types, features, limitations, and service lifetime against the current Microsoft documentation before production execution.

4. Critical 9 TB Timing Consideration

A major item to validate is the documented migration lifetime. Microsoft's migration best-practices guidance describes a maximum migration lifetime of seven days (168 hours), with additional cutover

timing constraints after the base copy completes. This makes throughput testing essential for a 9 TB database.

9 TB / 7 days ~ 15 MB/sec sustained

Real throughput is affected by table distribution, indexes, large objects, source I/O, network latency, concurrent workload, Azure target sizing, and post-copy object creation. A production-like rehearsal should demonstrate significant headroom. For an on-premises source, ExpressRoute or another stable high-bandwidth private path is generally preferable.

5. Logical Replication Readiness

Every large or frequently updated table should be checked for a primary key or suitable replica identity. Without usable row identity, UPDATE and DELETE replication can become problematic. REPLICA IDENTITY FULL can be a workaround but may increase WAL generation significantly.

```
SELECT
  n.nspname AS schema_name,
  c.relname AS table_name,
  c.reltuples::bigint AS estimated_rows
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE c.relkind = 'r'
AND n.nspname NOT IN ('pg_catalog', 'information_schema')
AND NOT EXISTS (
  SELECT 1 FROM pg_index i
  WHERE i.indrelid = c.oid AND i.indisprimary
)
ORDER BY estimated_rows DESC;
```

Adding a primary key to a very large production table can itself be substantial, so assess this well before the migration window.

6. Native PostgreSQL Logical Replication

If the Azure migration service does not fit the environment, native PostgreSQL logical replication is a strong alternative. Configure a publication on the source and a subscription on the target. Schema and non-table objects must be handled separately, and the DBA team owns more of the orchestration.

```
Source EDB PostgreSQL 15
  |
  | PUBLICATION
  v
Azure PostgreSQL Flexible Server
  |
  | SUBSCRIPTION
  v
Continuous synchronization
```

- Fine-grained control over migrated tables.
- Ability to organize very large tables into waves.
- Direct control of replication monitoring and cutover.
- Potential to preload historical/static data separately.
- More responsibility for schema, DDL, sequences, permissions, extensions, validation, retries, and recovery.

7. pglogical

pglogical is another logical-replication option and can be useful when more flexibility than standard publication/subscription behavior is required. Azure Flexible Server supports selected pglogical versions; verify the exact PostgreSQL 15 extension version in Microsoft's current extension matrix.

Suggested decision order: Azure Flexible Server Migration Service first; native logical replication if unsuitable; pglogical when additional replication flexibility is required.

8. Confirm the Exact EDB Product

Distinguish EDB-packaged community PostgreSQL from EDB Postgres Advanced Server (EPAS). Community PostgreSQL distributed by EDB is generally simpler. EPAS requires deeper compatibility assessment because it can contain Oracle-compatible packages, data types, SQL/SPL constructs, catalog behavior, and other EDB-specific features not automatically available on Azure Flexible Server.

Azure supports extensions such as orafce, but do not assume that an extension provides full EPAS/Oracle compatibility. EPAS migrations may require application and schema remediation in addition to data movement.

9. Extension Compatibility

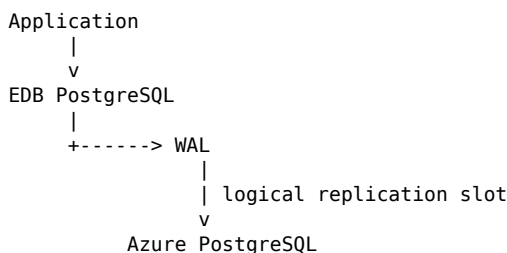
Inventory source extensions early:

```
SELECT
    extname,
    extversion
FROM pg_extension
ORDER BY extname;
```

Compare every installed extension with Azure Flexible Server's supported-extension list. Unsupported third-party extensions can become migration blockers.

10. WAL and Replication Slot Capacity Planning

For a 9 TB online migration, WAL retention is a major operational risk. While the initial database is copied, production transactions continue. Logical replication slots can retain WAL until the target consumes the changes.



If the source generates hundreds of GB of WAL per day and the initial copy takes several days, additional source storage requirements can become very large. Measure peak and average WAL generation and define alert thresholds.

```
SELECT pg_current_wal_lsn();

SELECT
    slot_name,
    active,
    restart_lsn,
    confirmed_flush_lsn
FROM pg_replication_slots;
```

WAL capacity planning should be a formal go/no-go prerequisite.

11. Sequence Synchronization

Logical replication primarily moves table changes; sequences require explicit attention depending on the migration mechanism. Before allowing writes on Azure, verify sequence values against migrated data.

```
SELECT setval(
    'orders_id_seq',
    (SELECT MAX(id) FROM orders)
);
```

For many sequences, automate and validate the synchronization. Incorrect values can cause duplicate-key failures immediately after cutover.

12. DDL and Deployment Freeze

Treat synchronization as a controlled database-change window. Logical CDC is not a general database-wide DDL replication solution.

- Avoid CREATE TABLE / DROP TABLE unless explicitly supported and planned.
- Avoid ALTER TABLE and column changes.
- Avoid partition restructuring.
- Control TRUNCATE usage.
- Freeze application releases that include schema changes.

13. Azure Target Sizing for Migration

Do not size the target only for steady-state production. Temporarily over-provision compute, IOPS, and storage headroom to maximize load and catch-up performance.

Migration window:

Compute -> high
IOPS -> high
Storage -> generous headroom
HA -> consider disabling during bulk load
Replicas -> disable during bulk load

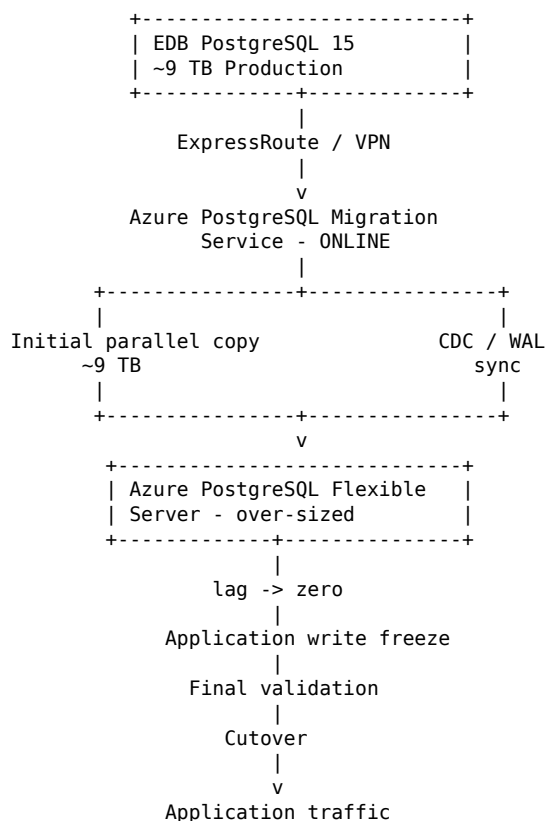
After migration:

Enable HA
Enable read replicas as required
Right-size compute
Confirm backup/retention policy
Configure monitoring and alerts

14. Pre-Migration Validation

Use the migration service's validation-only capability well before production. Resolve source/target readiness errors, unsupported objects, connectivity problems, and extension issues, then rerun until clean. Review known limitations carefully.

15. Recommended Migration Architecture



16. Practical Migration Runbook

1. Inventory schemas, tables, sizes, extensions, tablespaces, PKs, sequences, large objects, collations, roles, permissions, and EDB/EPAS-specific objects.
2. Confirm whether the source is community PostgreSQL packaged by EDB or EPAS.
3. Measure normal and peak WAL generation.
4. Run Azure pre-migration validation and resolve blockers.
5. Build a production-sized Azure test target.
6. Establish and benchmark private network connectivity.
7. Configure and validate logical-replication prerequisites and WAL capacity.
8. Run a full 9 TB rehearsal if possible, or a representative multi-terabyte rehearsal.
9. Measure TB/hour, source impact, lag, WAL retention, and catch-up behavior.
10. Tune source, network, target SKU, IOPS, and migration parallelism.
11. Rebuild or clean the target for production migration.
12. Start online migration and complete the initial bulk load.
13. Allow CDC to catch up while monitoring lag and retained WAL.
14. Enforce DDL/application deployment freeze.
15. Stop application writes for final cutover.
16. Wait until replication lag is effectively zero.
17. Synchronize and validate sequences.
18. Run database, table, checksum/sample, and business-level validation.
19. Switch DNS/connection strings/secrets to Azure.
20. Perform read/write smoke tests, then open application traffic.
21. Enable/configure Azure HA, replicas, backup policy, monitoring, alerts, and normal operations.
22. Retain the source according to rollback and decommissioning policy.

17. Validation Strategy

Validation level	Examples
Database / schema	Database, schema, table, index, function, view, and object counts
Table	Row counts and partition counts
Critical tables	PK ranges, MIN/MAX, SUM, sampled checksums
Business	Balances, transaction totals, order counts, operational reports
Functional	Critical read/write smoke tests
Performance	Top SQL, latency, CPU, I/O, connections, locks, query-plan comparison

18. Key Risks and Mitigations

Risk	Impact	Mitigation
Migration exceeds service lifetime	Migration fails or must restart	Benchmark early; over-provision; verify limits with Microsoft
High WAL generation / slot retention	Source storage exhaustion	Measure WAL; add headroom; monitor slots and lag
Tables without PK/replica identity	Incorrect UPDATE/DELETE replication	Add PK or use tested replica identity strategy
Unsupported EDB/EPAS features	Schema/application incompatibility	Compatibility assessment and remediation

Risk	Impact	Mitigation
Unsupported extensions	Migration blocker/runtime failure	Map each extension to Azure-supported alternatives
Sequence mismatch	Duplicate keys after cutover	Synchronize and validate before writes
DDL during migration	Schema/data divergence	Enforce change freeze
Under-sized target	Slow copy/catch-up	Scale compute/IOPS; remove unnecessary overhead
Insufficient validation	Undetected data/business issues	Technical and business reconciliation

19. Go/No-Go Checklist

- Azure pre-migration validation passes.
- All required extensions are supported or remediated.
- EPAS-specific dependencies are identified and remediated, if applicable.
- Tables without primary keys have an approved replication strategy.
- Network throughput has been benchmarked.
- 9 TB duration fits comfortably inside the supported service window.
- WAL generation and source storage headroom are understood.
- Target compute, storage, and IOPS are sized for migration load.
- Sequence synchronization procedure is tested.
- DDL/application release freeze is approved.
- Data and business validation scripts are ready.
- Cutover and rollback procedures are documented and rehearsed.
- Application connection-string/DNS change is tested.
- Operations team is ready for post-cutover monitoring.

20. Documentation and Reference Links

Azure PostgreSQL migration service overview

<https://learn.microsoft.com/en-us/azure/postgresql/migrate/migration-service/overview-migration-service-postgresql>

Online migration from PostgreSQL on-premises/VMs

<https://learn.microsoft.com/en-us/azure/postgresql/migrate/migration-service/tutorial-migration-service-iaas-online>

Migration service best practices

<https://learn.microsoft.com/en-us/azure/postgresql/migrate/migration-service/best-practices-migration-service-postgresql>

Pre-migration validation

<https://learn.microsoft.com/en-us/azure/postgresql/migrate/migration-service/concepts-premigration-migration-service>

Migration service known issues / limitations

<https://learn.microsoft.com/en-us/azure/postgresql/migrate/migration-service/concepts-known-issues-migration-service>

Azure PostgreSQL supported extensions

<https://learn.microsoft.com/en-us/azure/postgresql/extensions/concepts-extensions-versions>

Creating/managing PostgreSQL extensions on Azure

<https://learn.microsoft.com/en-us/azure/postgresql/extensions/how-to-create-extensions>

Azure DMS PostgreSQL online known issues

<https://learn.microsoft.com/en-us/azure/dms/known-issues-azure-postgresql-online>

Azure PostgreSQL Flexible Server limits

<https://learn.microsoft.com/en-us/azure/postgresql/configure-maintain/concepts-limits>

EDB Oracle compatibility documentation

https://www.enterprisedb.com/docs/edb-postgres-ai/databases/oracle_compatibility/

21. Final Recommendation

For EDB PostgreSQL 15 (~9 TB) to Azure Database for PostgreSQL Flexible Server, use Azure Flexible Server Migration Service in ONLINE mode as the primary design, subject to a successful production-scale throughput rehearsal and confirmation that the migration fits within current service limits. Use native PostgreSQL logical replication as the main fallback when the managed service's time window, object restrictions, or source-specific requirements make it unsuitable. Consider pglogical when additional replication flexibility is needed.

The two most important facts to establish early are: (1) whether the source is community PostgreSQL packaged by EDB or EDB Postgres Advanced Server, and (2) the source's normal and peak WAL/change rate.